Machine Learning Assignment-1

Decision Trees, Bayesian Classification, and Evaluation

Deadline: 6 September 2026, 11:59 PM
Total Marks: 100
Resources: Open book, open internet, open code, open LLM

General Instructions

All students receive exactly the same questions, datasets, numerical constants, starter code, deadline, and grading rubric. You may use textbooks, papers, online resources, code assistants, and generative AI systems. The purpose of this assignment is therefore not to test recall of definitions or syntax.

Marks are awarded for mathematical correctness, algorithmic correctness, reproducibility, adversarial testing, and the ability to justify why a result is true. A plausible-looking answer that cannot be reproduced or that fails the common hidden test suite will not receive full credit.

For questions involving words such as **smallest**, **largest**, **minimum**, **maximum**, **unique**, or **impossible**, you must submit an executable computational certificate supporting the claim. A numerical search without a mathematical argument is insufficient where a proof is requested.

Unless explicitly permitted, do not use pre-built implementations of CART, QDA, Naive Bayes, ROC-AUC, average precision, F1 optimization, or confusion-matrix metrics from machine-learning libraries. Basic NumPy linear algebra and sorting operations are allowed.

For any clarification refer to the Readme.md supplied with this Assignment.

Common Hidden Evaluation

The TA's will run every student's submitted functions on the **same hidden test suite** after submission. Hidden cases may include duplicated feature values, exact ties, all-constant columns, missing values, near-singular covariance matrices, severe class imbalance, repeated score values, and extreme numerical scales. The hidden test cases are identical for all students.

Question 1 - Reverse Engineer a Decision Tree (12 marks)

A depth-3 binary CART classifier using Gini impurity is trained on the supplied dataset. You are given the dataset and the final prediction vector, but not the learned tree.

For a node containing sample set S, use

$$G(S) = 1 - \sum_{k \in \{0,1\}} p_k^2.$$

A legal threshold for feature j is exactly

$$t = \frac{v_i + v_{i+1}}{2},$$

where v_i and v_{i+1} are consecutive distinct finite values of that feature present at the node. A split is illegal if either child has fewer than three observations.

The impurity decrease is

$$\Delta G = G(S) - \frac{|S_L|}{|S|} G\left(S_L\right) - \frac{|S_R|}{|S|} G\left(S_R\right).$$

Use deterministic tie-breaking in this order: larger ΔG, smaller feature index, smaller threshold, then missing-left before missing-right.

1(a) Implement CART from scratch and recover the exact tree. Report every candidate root split, sorted from best to worst, with

$$\left(j, t, m, n_L, n_R, G_L, G_R, \Delta G\right),$$

where $m \in \{L, R\}$ denotes the missing-value direction.

1(b) Remove exactly one training observation so that the feature selected at the root changes. Among all observations with this property, return the smallest valid index.

Question 2 - Make Greedy CART Fail as Badly as Possible (13 marks)

Using only binary-valued features, construct the **smallest possible labelled dataset** for which all of the following hold:

1. greedy depth-2 CART chooses a unique root split;
2. the resulting greedy tree has non-zero training error;
3. another depth-2 tree achieves zero training error.

You must determine the minimum possible number of observations n.

Your submission must contain the dataset, the greedy tree, the globally optimal tree, the two training errors, a proof that the greedy root is unique, and a minimality certificate establishing that no smaller dataset works.

You may use exhaustive enumeration as part of the certificate, but your program must independently verify every size $1, 2, ..., n - 1$.

Question 3 - Can One Point Destroy a Tree? (10 marks)

For the supplied dataset, determine whether there exists one additional training observation

$$\left(x^\star, y^\star\right)$$

such that all three conditions hold:

1. the original root feature is no longer selected;
2. the training accuracy of the resulting depth-2 tree decreases;
3. the newly added observation is nevertheless classified correctly by the resulting tree.

If such a point exists, construct one minimizing

$$\| x^\star \|_2.$$

If no such point exists, prove impossibility under the stated split rules. A finite grid search does not constitute a proof of optimality or impossibility.

Question 4 - Implement Naive Bayes

Implement the Naive Bayes Algorithm taught in the class. You are not allowed to use any library except numpy.

Question 5 - The Classifier Ranking Paradox (12 marks)

Construct the **smallest possible binary classification dataset** and predictions from three classifiers A, B, and C such that

$$Accuracy(A) > Accuracy(B) > Accuracy(C),$$

but simultaneously

$$F_1(C) > F_1(B) > F_1(A).$$

Additionally require

$$MCC(B) > MCC(A)$$

and

$$BalancedAccuracy(A) < BalancedAccuracy(C).$$

Determine the smallest possible number of observations n.

You may use exhaustive search. Full credit requires a minimality certificate demonstrating that no valid construction exists for any smaller n.

For the final construction, report the label vector, all three prediction vectors, all three confusion matrices, and the values of Accuracy, Precision, Recall, F_1, MCC, and Balanced Accuracy.

Question 6 - Same ROC-AUC, Completely Different Behaviour (10 marks) (Follow up of Question 6)

Construct two score vectors s_A and s_B on the same labelled dataset satisfying

$$AUC(A) = AUC(B),$$

while maximizing

$$\left|F_1(A) - F_1(B)\right|$$

when both score vectors are thresholded at 0.5.

The scores must satisfy

$$0 \leq s_i \leq 1,$$

and the construction must additionally satisfy

$$AP(A) \neq AP(B).$$

You must find the optimum rather than merely provide one example. Give both score vectors, the ROC and precision-recall curves, confusion matrices at threshold 0.5, AUC, AP, F_1, and a proof or exhaustive certificate establishing optimality under the supplied dataset-size bound.

Your ROC-AUC implementation must correctly account for tied scores, using

AUC = P($s_+ > s_-$) + ½ P($s_+ = s_-$).

Question 7 - Find the Bugs (10 marks)

You will receive approximately 80 lines of apparently reasonable Python code implementing parts of CART, QDA, ROC-AUC, and F_1 evaluation. The program runs successfully and produces plausible-looking outputs, but it contains exactly **six conceptual bugs**.

For each bug, submit the following six items:

1. the faulty line or logical block;
2. the smallest input you can establish that exposes the bug;
3. the observed incorrect output;
4. the correct output;
5. the mathematical or algorithmic reason the implementation is wrong;
6. a minimal patch.

Full credit for a bug requires an input that actually distinguishes the faulty implementation from the correct one. Merely pointing to suspicious style or inefficient code receives no credit.

Submission Requirements

Submit exactly the following files:

```
<student-id>/
|-- derivations.pdf
|-- solution.py
|-- results.json
|-- certificates/
`-- reproduce.sh
```

Running

```
bash reproduce.sh
```

must regenerate all reported numerical results, tables, counterexamples, certificates, and figures from the supplied common assignment data.

Every numerical result appearing in derivations.pdf must be reproducible from submitted code.

Required Implementation Principles

- Do not hard-code final outputs.
- Do not identify test cases by filenames or data dimensions.
- Handle ties explicitly and deterministically.
- Avoid numerically unstable probability-domain calculations when a log-domain formulation exists.
- State every convention used for undefined metrics such as precision or F_1 when denominators are zero.
- The same hidden tests and grading rubric will be used for every student.

Verification Viva

After submission, each student may be asked to explain or modify two randomly selected parts of their own solution. Typical settings include predicting what happens after a label flip, explaining why a threshold is legal, identifying the first place two trees diverge, or constructing an input that breaks a specific line of code.

The viva is intended only to verify understanding of the submitted work; all students are evaluated under the same procedure.
